

Assignment 3 | Bridge Pattern

Sungat Serikov

SE-2528

D — Remote controls

https://github.com/SungatSerikov/Assignment3_SDP

1. Design and choice of Bridge

This project uses the Bridge pattern to connect remote controls with devices. There are two independent dimensions: remote types and device types.

BasicRemote turns a device on with volume 30, while QuietRemote turns it on with volume 5. The available devices are TvDevice, RadioDevice, and ProjectorDevice.

Remote is the abstraction, and Device is the implementor interface. Each remote receives a device through its constructor and stores it in the interface-typed field implementation. The execute() method delegates to Device.applySettings().

Bridge fits this project because both hierarchies can grow independently while using the same interface [1]. Adding a device does not require changing the existing remote classes. The client, Main, selects the concrete combinations.

Although Remote is abstract, its execute() method is concrete. Both remote subclasses inherit this method and supply different volume presets.

2. UML class diagram

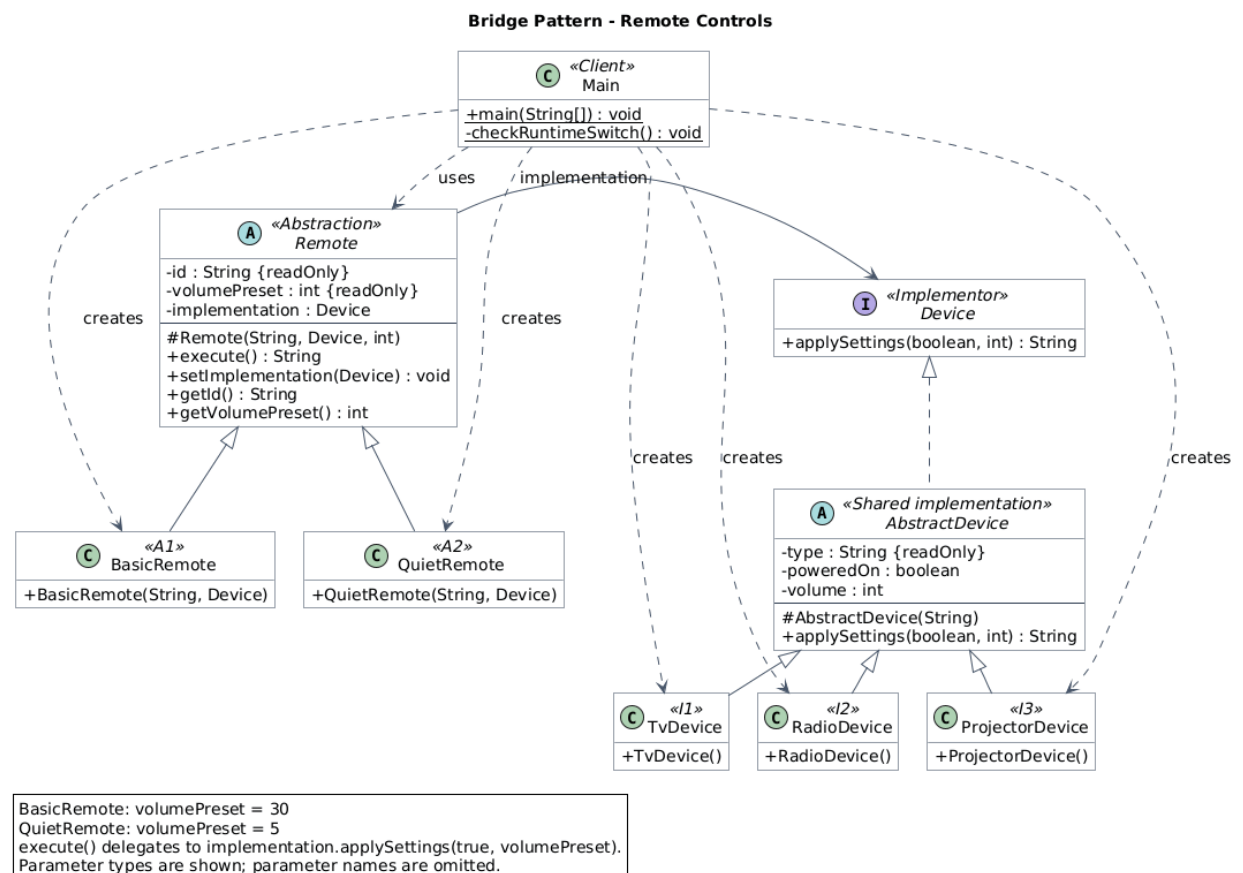


Figure 1. Remote and device hierarchies connected through the Device interface.

BasicRemote and QuietRemote extend Remote. AbstractDevice implements Device, and all three concrete devices extend AbstractDevice.

The Remote → Device association represents the stored device reference. Dashed dependencies show that Main uses Remote and creates the concrete objects. The dashed line with a hollow triangle from AbstractDevice to Device represents interface realization.

Remote and AbstractDevice are abstract classes. Underlined methods in Main are static.

3. Clean Code examples

3.1 Separation of responsibilities

Source: src/Remote.java

```
public String execute() { 3 usages  Sungat Serikov
    return this.implementation.applySettings( poweredOn: true, this.volumePreset);
}
```

The remote selects the power setting and volume preset, then delegates to the device. The device manages its state and produces the result. This separation keeps each class focused on its responsibility.

3.2 Meaningful names

Source: src/Device.java

```
public interface Device { 7 usages 4 implementations  Sungat Serikov
    ⚡ String applySettings(boolean poweredOn, int volume); 1 usage
}
```

The method and parameter names explain their purpose. applySettings describes the operation, poweredOn identifies the power state, and volume identifies the requested volume. These names make the interface easier to understand.

3.3 Small focused methods

Source: src/Remote.java

```
public void setImplementation(Device implementation) {
    this.implementation = implementation;
}
```

This method performs one action: replacing the device reference. Its limited responsibility makes it easy to read, check, and use in the runtime-switch demonstration.

3.4 Avoidance of duplication

Source: src/AbstractDevice.java

```
@Override 1 usage Sungat Serikov
public String applySettings(boolean poweredOn, int volume) {
    this.poweredOn = poweredOn;
    this.volume = volume;

    return this.type + " power="
        + (this.poweredOn ? "ON" : "OFF")
        + " volume=" + this.volume;
}
```

TV, radio, and projector inherit the same state-update and output logic. Keeping this behavior in `AbstractDevice` avoids repeating it in three classes. Changes to the shared behavior can be made in one place.

3.5 Encapsulation of state

Source: `src/Remote.java`

```
public abstract class Remote { 6 usages 2 inh
    private final String id; 2 usages
    private final int volumePreset; 3 usages
    private Device implementation; 3 usages
```

Private fields prevent direct access from outside the class. The final fields preserve the remote ID and volume preset after construction. The device reference can be replaced through `setImplementation()`. This supports the preservation of remote data checked in T5.

4. Independent extension and demonstration

The base commit is:

c311bca018d6bf5190e9cf11bae25d2b320ea429

This version contains TV, radio, and checks T1–T5. The extension adds `ProjectorDevice` and checks T6–T7.

Within `src/`, the only changes are the new `ProjectorDevice.java` and updates to `Main.java`. Existing remote classes, `Device`, `AbstractDevice`, `TvDevice`, and `RadioDevice` remain unchanged. `sources.txt` includes the new file, and `extension.diff` records the source changes.

T1–T4 check both remote types with TV and radio. In T5, the same `BasicRemote` switches from TV to radio. Object identity is checked using `==`; its ID remains `SWITCH-1`, and its volume preset remains 30. The output changes from `TV power=ON volume=30` to `RADIO power=ON volume=30`.

T6–T7 check both remote types with the projector. All seven checks passed:

SUMMARY: 7/7 PASS

PASS or FAIL is calculated from actual comparisons. The captured results are stored in `demo-output.txt`.

5. Trade-off of Bridge

Bridge introduces additional classes and indirection. Understanding one operation requires following the remote, the interface, and the shared device implementation. The flexibility therefore comes with additional structural complexity, which is more noticeable in a small program.

6. Bridge versus Adapter

Bridge separates abstraction and implementation so that both dimensions can develop independently. In this project, remote types and device types share the Device contract.

Adapter makes an existing incompatible interface usable by a client [1]. It wraps an existing object and exposes the interface the client expects. The distinction is their intent: Bridge supports independent variation, while Adapter solves interface incompatibility.